## Task 7: Automation for Temp Directory File Count Monitoring (AWS Lambda → Pushgateway → Grafana)

### Objective

Build an lambda monitoring solution that:

- Counts the number of files in a system temp directory (example: `C:\Users\User\AppData\Local\Temp`)
- Publishes this metric to **Grafana** via a **Pushgateway**
- Visualizes the metric in **Grafana dashboards**
- Sends an **email alert** if the file count exceeds **100**

**Bonus:** Containerize the collector and deploy it on both:

1. **AWS Lambda (container image)**
2. **ECS Fargate**

---

### High-Level Architecture

1. **Collector (Lambda)** runs on a schedule (e.g., every 5 minutes).
2. Collector computes: `temp_dir_file_count`.
3. Collector pushes metric to **Pushgateway** (HTTP endpoint).
4. **Prometheus** (or Grafana Agent / compatible scraper) scrapes Pushgateway.
5. **Grafana** reads from Prometheus data source.
6. **Grafana Alerting** triggers email when `temp_dir_file_count > 100`.

---

### Prerequisites

- AWS account access with permission to deploy Lambda, IAM roles/policies, scheduler (EventBridge), ECS (for bonus)
- Grafana instance available (Cloud or self-managed)
- Pushgateway reachable from the collector (networking + firewall rules)
- Prometheus (or equivalent) scraping Pushgateway and configured as Grafana data source
- Email channel configured in Grafana (SMTP or Grafana Cloud alerting contact point)

---

## Task Requirements

#### Part 1: Build the Collector (Lambda)

- Implement a lightweight collector that:
  - Reads a configured temp directory path (configurable input)
  - Counts files in that directory
  - Produces a numeric metric: `temp_dir_file_count`
- Ensure the collector includes:
  - Minimal logging (success/failure + count)
  - Basic error handling (directory missing, permission denied, push failure)

**Config should be externalized**, e.g. environment variables:

- `TEMP_DIR_PATH`
- `PUSHGATEWAY_URL`
- `METRIC_JOB_NAME` (e.g., `tempdir-monitor`)
- Optional labels like `host`, `environment`, `yourname`

---

#### Part 2: Automate Execution (Schedule)

- Run the Lambda automatically using a scheduler (e.g., EventBridge rule).
- Frequency should be reasonable (e.g., every 1–5 minutes).

---

#### Part 3: Push Metric to Pushgateway

- Push the metric to the Pushgateway in Prometheus-compatible format.
- Use a consistent job name and labels so the time series is stable.

---

#### Part 4: Grafana Dashboard

Create a Grafana dashboard that includes:

- A time-series panel showing `temp_dir_file_count` over time
- A stat panel showing “Current temp dir file count”
- Optional: thresholds (green under 100, red above 100)

---

#### Part 5: Alerting (Email if Count > 100)

Configure a Grafana alert rule:

- Condition: `temp_dir_file_count > 100` (sustained for a short window, e.g., 5–10 minutes to avoid noise)
- Send notification to email
- Include key details in the alert message:
  - Directory path
  - Current count
  - Timestamp
  - Labels (host/environment)

---

### Bonus Task: Containerize + Dual Deployment

#### Bonus A: Containerize the Collector

- Package the collector into a container image
- Keep it minimal (small base image, single responsibility)

#### Bonus B: Deploy Container to Lambda

- Deploy the container image to Lambda (container-based Lambda)
- Keep the same behavior and configs via environment variables

#### Bonus C: Deploy Container to ECS Fargate

- Deploy the same container to ECS Fargate as a scheduled task (or service + scheduled run)
- Ensure it can reach Pushgateway and push the metric
- Use CloudWatch logging

---

### Validation Checklist

Verify:

- Collector runs on schedule and logs the file count
- Pushgateway receives the metric (metric visible via scrape endpoint / Prometheus)
- Grafana dashboard shows data updating over time
- Alert triggers when count exceeds 100 (test by simulating higher count)
- Email notification is received with correct details
- (Bonus) Container runs successfully on Lambda
- (Bonus) Container runs successfully on ECS Fargate

---

**Expected Deliverables**

Submit the following:

1. Requirement analysis (Confluence link)
2. Technical documentation (Confluence link)
3. Prepare a short document explaining difference between containerization in Lambda and ECS . (Confluence link)
    - Also include a short video (2–3 mins) explaining the same documentation.